MCPs: construye tu propio servidor MCP

Master AI Engineer · Sesión 15 · Ignacio de Pastors

thePower Tech School

AGENDA

01 Bloque teórico

02 Bloque práctico

03 Ejercicio para casa

04 Cierre y dudas



thePower Tech School

01

Bloque teórico

thePower
Tech School

Conectar un LLM a cada herramienta era un caos a medida

EL PROBLEMA — antes de MCP

01 **Integración a medida**
Cada herramienta (Drive, Jira, Postgres) exigía su propio conector y su propio código.

02 **N x M plugins**
3 clientes x 10 herramientas = 30 integraciones que mantener y actualizar.

03 **Cambias de modelo, reescribes**
Pasar de un modelo a otro obligaba a rehacer los conectores desde cero.

04 **No escala: trabajo duplicado**
Cinco equipos construyendo cinco conectores distintos para el mismo Slack.

ANALOGÍA

Un cargador distinto para cada aparato: un cajón de cables que no sirven entre sí.

Caso real: la misma empresa usa Claude en negocio y Cursor en desarrollo, y mantiene dos conectores a Jira que hacen exactamente lo mismo.

thePower Tech School

MCP: un estándar abierto, el USB-C de la IA

QUÉ ES MCP — el estándar que lo ordena

01 **Estándar abierto y público**
Lo publicó Anthropic en noviembre de 2024; la especificación es abierta y cualquiera puede implementarla.

02 **Un único idioma común**
Modelo y herramienta hablan el mismo protocolo: misma forma de descubrir, pedir y devolver datos.

03 **Construyes el servidor UNA vez**
Un servidor MCP de tu CRM se escribe una sola vez, no un proyecto por cada cliente de IA.

04 **Compatible con cualquier cliente**
Claude, Cursor, VS Code... Pasas de N x M integraciones a N + M conectores.

IDEA CLAVE

MCP es a las herramientas de IA lo que el USB-C a los cables: un conector universal.

Caso real: levantas un servidor MCP de GitHub y ese mismo servidor funciona igual en Claude y en Cursor, sin tocar una línea de código.

thePower Tech School

API vs MCP: no compiten, una envuelve a la otra



LA API, TAL CUAL lo que lee un programador

```
GET https://api.frankfurter.dev/v1/latest
    ?from=EUR&to=USD
→ {"base": "EUR", "date": "2026-09-25",
   "rates": {"USD": 1.1403}}
```

Tienes que leer su documentación y escribir el código. Cada API tiene su URL, sus parámetros y su formato.

LA MISMA FUNCIÓN, COMO TOOL MCP lo que ve el modelo (tools/list)

```
name: "convertir_precio"
description: "Convierte un importe en euros a otra
  moneda con el tipo de cambio oficial del BCE de hoy."
inputSchema: importe_eur (number), moneda (string)
```

Se describe sola, con el mismo formato que cualquier otro servidor. El modelo la descubre y decide cuándo usarla.

	API	MCP
Pensada para	programadores que leen su documentación	**modelos que descubren las tools solos**
Idioma	cada una el suyo: URL, parámetros, formato	**el mismo protocolo para todas (JSON-RPC)**
Quién la llama	tu código, donde tú lo escribas	**el modelo, cuando la pregunta lo pide**
Si cambias de host	reescribes la integración	**la enchufas tal cual en otro host**

IDEA CLAVE La API es el servicio; MCP es el USB-C que le pones para que cualquier agente pueda usarlo.

thePower Tech School

Tres piezas: host, cliente y servidor

01

HOST

La aplicación con la que hablas y que orquesta toda la conversación.

Ej.: Claude Desktop, Claude Code o VS Code.

02

CLIENTE

Vive dentro del host y mantiene UNA conexión con UN servidor.

Ej.: 3 servidores conectados = 3 clientes aislados.

03

SERVIDOR

El programa que construyes TÚ y que expone las capacidades.

Ej.: un servidor que lee tu Postgres y devuelve las ventas.

Como en un restaurante: tú pides del menú (cliente), la cocina prepara el plato (servidor) y la sala coordina que todo llegue a la mesa (host).

El host crea un cliente por cada servidor. Todos los mensajes viajan en JSON-RPC 2.0: el mismo formato para todos, y el SDK lo gestiona por ti.

thePower Tech School

Un servidor MCP expone tres primitivas

01

TOOLS

Funciones que el modelo EJECUTA.
Las elige el modelo según lo que pide el usuario.

Ej.: buscar en la web, enviar un email, crear un issue en GitHub.

Son los VERBOS del servidor

02

RESOURCES

Datos que el modelo LEE.
Solo lectura, identificados por una URI (su dirección).

Ej.: un fichero de logs, la ficha de un cliente, un informe.

Como un GET: informan, no cambian nada

03

PROMPTS

Plantillas de conversación reutilizables.
Las elige el USUARIO, no el modelo.

Ej.: 'revisa este PR' o 'resume la reunión', listos para usar.

Son los atajos del equipo

En clase nos centramos en TOOLS: cubren el 80% de los casos reales. Resources y Prompts quedan de referencia para cuando los necesites.

thePower Tech School

Tools: el corazón de tu servidor MCP

TOOLS EN DETALLE — de función a herramienta

01

Una función Python con @mcp.tool()

Decoras la función y el SDK la publica como herramienta: no escribes esquemas a mano.

02

Type hints + docstring = definición

buscar_pedidos(fecha: str, limite: int = 5) con su docstring ya define la tool entera.

03

El modelo decide cuándo llamarla

Tú pides 'dame los pedidos de ayer' y él elige la tool y rellena los parámetros.

04

Valida entradas y controla errores

Mejor devolver 'fecha no válida, usa AAAA-MM-DD' que una excepción: así el modelo reintenta.

IDEA CLAVE

Escribir un buen docstring ES escribir tu API: el modelo elige por lo que lee ahí.

Caso real: una tool llamada buscar() casi nunca se usa; buscar_pedidos_por_fecha(), con su docstring explicando qué devuelve, se usa siempre.

thePower Tech School

Transporte: STDIO en local, HTTP en la nube

EL TRANSPORTE — cómo viajan los mensajes

01

STDIO: subproceso local

El host arranca tu servidor en tu máquina y hablan por stdin/stdout, como al encadenar comandos.

02

Cero red, cero puertos

Nada que abrir ni desplegar: es el transporte por defecto de Claude Code y el más rápido de probar.

03

HTTP Streamable: remoto

Para un servidor en la nube que sirve a muchos clientes a la vez, con autenticación y sesiones.

04

Hoy usamos STDIO

Regla práctica: local y para ti, STDIO; compartido por el equipo o en la nube, HTTP.

OJO

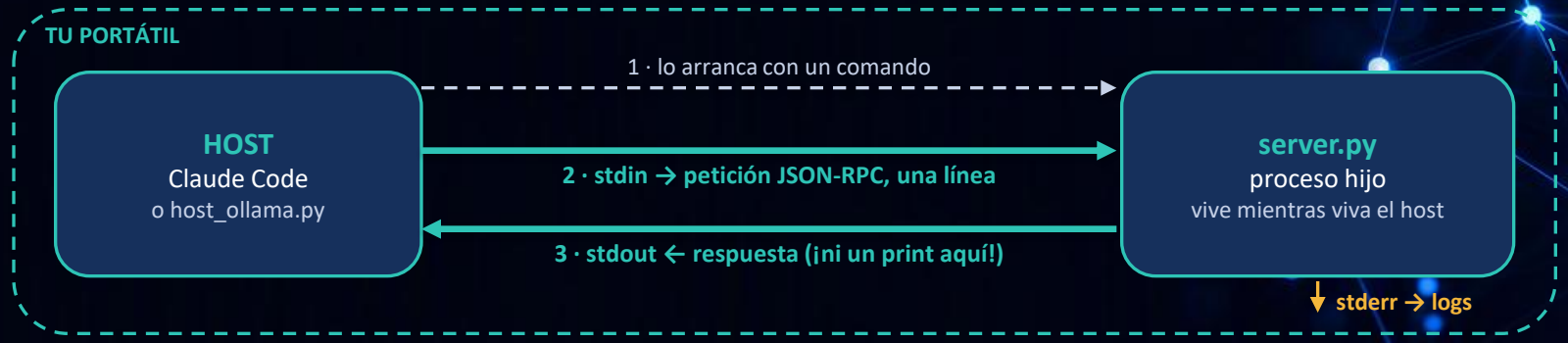
En STDIO NUNCA uses print(): corrompe los mensajes JSON-RPC. Usa logging a stderr.

Error número uno del principiante: un print() de depuración suelto y el servidor deja de responder sin dar ninguna pista.

thePower Tech School

STDIO vs HTTP: el mismo servidor, dos caminos

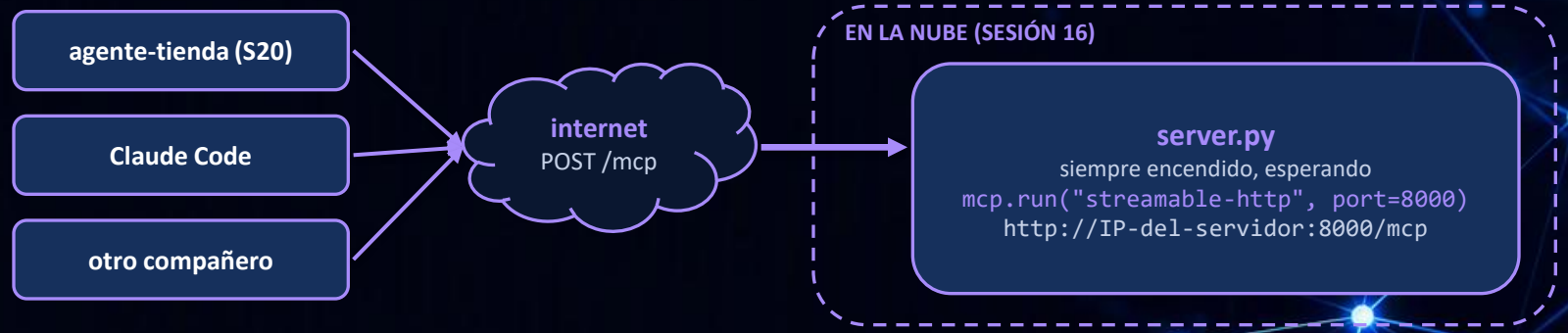
STDIO · el host arranca tu servidor y le habla por un tubo privado



SE CONECTA CON un comando
`python server.py`

- 1 host = 1 proceso = 1 cliente
- Sin red, sin puertos, sin login
- **Lo que usamos hoy**

HTTP STREAMABLE · tu servidor vive en su casa, con una dirección, y cualquiera que la sepa puede llamar



SE CONECTA CON una URL
`Client("http://…:8000/mcp")`

- Muchos hosts a la vez, con sesión
- Hay que desplegar, abrir puerto y proteger
- **Real: conectar 193 ms, una tool 57 ms**

IDEA CLAVE Las tools no cambian: cambia cómo se llega a ellas. STDIO es un tubo privado; HTTP, una dirección.

thePower Tech School

Un mensaje, dos viajes: así se envía un tools/call

EL MENSAJE ES EL MISMO · JSON-RPC 2.0

```
{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "consultar_pedido", "arguments": {"numero": "1234"}}}
```

STDIO · una línea por un tubo

1 · El host escribe en el stdin del proceso

```
{"jsonrpc":"2.0","id":3,"method":"tools/call",…}
\n   ← el salto de línea marca el final
```

2 · El servidor contesta con UNA línea por stdout

```
{"jsonrpc":"2.0","id":3,"result":{"content":[…
  "estado": "enviado", "transportista": "SEUR"…}
```

3 · Por qué rompe un print()

Escribe en el mismo tubo una línea que no es JSON. El host la lee como respuesta, no la entiende y la conexión se cuelga, sin error claro.

HTTP STREAMABLE · una petición web a /mcp

1 · El host hace un POST a la URL

```
POST /mcp
Content-Type: application/json
Accept: application/json, text/event-stream
Mcp-Session-Id: …  ← se lo dio el servidor al conectar
{ el mismo JSON de arriba }
```

2 · El servidor responde: JSON directo o un stream

```
200 OK · Content-Type: text/event-stream
data: {"jsonrpc":"2.0","id":3,"result":{…}}
```

3 · ¿Por qué «streamable»? Puede mandar avisos de progreso antes del resultado, por la misma respuesta.

IDEA CLAVE JSON-RPC es el idioma; STDIO y HTTP son solo el cable por el que viaja.

thePower Tech School

Cara a cara: cuándo STDIO y cuándo HTTP

	STDIO	HTTP STREAMABLE
Quién lo arranca	el host, como proceso hijo	tú lo despliegas; siempre encendido
Cómo se conecta	un comando: python server.py	una URL: http://…:8000/mcp
Cuántos clientes	uno por proceso	muchos a la vez, cada uno con su sesión
Seguridad	la de tu máquina: nada expuesto	puerto abierto: autenticación y HTTPS
Depurar	logging a stderr, nunca print()	logs del servidor y códigos HTTP
Úsalo para	tus herramientas locales y probar	el equipo, producción, agentes en la nube
En el máster	**S15: Claude Code y host_ollama.py**	**S16 en la nube · S18 en CI · S20 agente-tienda**

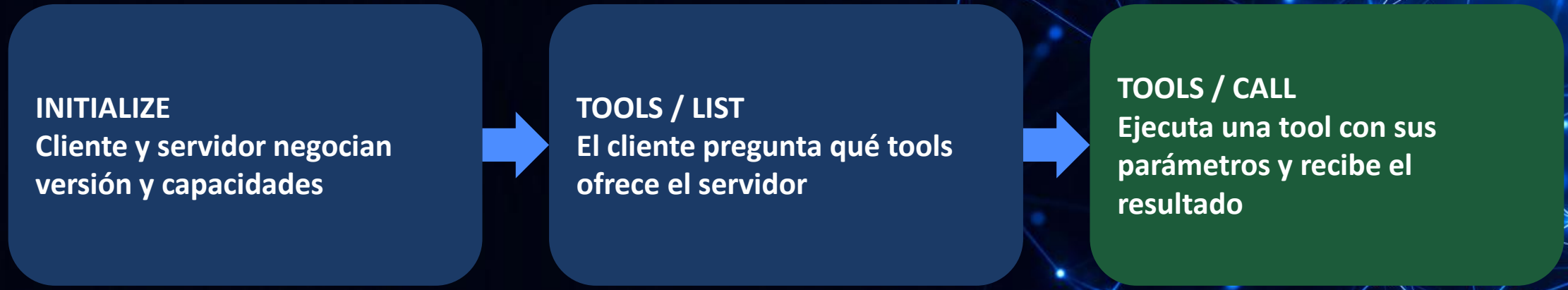
UN SOLO server.py PARA LOS DOS (así lo dejamos en mcp-server-ci, sesiones 16 y 18)

```
if MCP_TRANSPORT == "http":  mcp.run("streamable-http", host="0.0.0.0", port=8000)
else:                        mcp.run()   # stdio, por defecto
```

OJO En tutoriales antiguos verás «HTTP + SSE»: está obsoleto. Hoy el transporte remoto es Streamable HTTP.

thePower Tech School

El ciclo de vida: initialize, list y call



El SDK genera la definición de cada tool a partir de tus type hints y tu docstring.

thePower
Tech School

Buenas prácticas: claridad, seguridad y logging

BUENAS PRÁCTICAS

- Nombres y docstrings claros y descriptivos
- Define las entradas con tipos (JSON Schema)
- Valida y sanea todo lo que entra
- Logging a stderr, nunca a stdout

IDEA CLAVE

Un buen servidor MCP es pequeño, seguro y fácil de extender. La calidad está en los detalles.

thePower Tech School

02

Bloque práctico

thePower
Tech School

Qué hacemos hoy: un servidor MCP de verdad, en 5 pasos

1

ESQUELETO

proyecto uv + MCPServer que arranca por STDIO

`pasos/01_esqueleto.py`

2

PRIMERA TOOL

@mcp.tool() que llama a una API pública sin clave

`pasos/02_primera_tool.py`

3

CONECTAR

registrarlo en Claude Code y ver tools/list

`claude mcp add`

4

AMPLIAR

un resource + una segunda tool con validación

`server.py`

Paso 5: lo probamos de punta a punta con **un host casero con llama3.1.**

Sin claves, sin pago, todo local por STDIO. Y un cliente 'a pelo' (cliente_jsonrpc.py) que **enseña cada mensaje JSON-RPC que viaja.**

thePower Tech School

El caso: la tienda de la Sesión 5, como servidor MCP

mi-servidor

`TOOL` `convertir_precio(importe_eur, moneda)`
API pública del BCE (Frankfurter), sin clave

`TOOL` `consultar_pedido(numero)`
los pedidos 1234 y 5678 de la Sesión 5

`RESOURCE` `tienda://politica-devoluciones`
un fichero de texto, solo lectura

1. **Tools = verbos**
   el modelo decide llamarlas
2. **Resource = datos**
   la app decide leerlos, por URI
3. **Un solo servidor**
   vale para Claude Code, VS Code, Cursor...

En la Sesión 5 la tool consulta_pedido vivía DENTRO del agente. Hoy la sacamos a un servidor: **cualquier host la puede usar sin copiar código.**

thePower Tech School

Qué construimos: una pregunta, tres piezas

CLIENTE DE LA TIENDA
«¿Cuándo llega mi pedido 1234 y cuánto es en dólares?»

HOST + LLM
Claude Code, o host_ollama.py con llama3.1
Lee la pregunta y decide qué tools llamar

LO QUE BUSCAMOS
«Llega el jueves con SEUR. Son 49,90 €, unos 56,90 USD.»

mi-servidor (server.py) · **EL SERVIDOR MCP QUE ESCRIBIMOS HOY**

TOOL `consultar_pedido(numero)`
lee TUS datos
→ enviado · SEUR · jueves · 49,90 €

TOOL `convertir_precio(importe_eur, moneda)`
llama a una API DE FUERA
→ 49,90 € = 56,90 USD

RESOURCE `tienda://politica-devoluciones`
un documento de solo lectura
→ «30 días para devolver…»

Pedidos de la tienda
un diccionario con el 1234 y el 5678; en la vida real, tu base de datos

API Frankfurter
tipos de cambio oficiales del BCE, gratis y sin clave

politica_devoluciones.txt
un fichero de texto en datos/

Dato real: 56,90 USD con el tipo 1,1403 del BCE (25-09-2026). En clase, llama3.1 no encadenó las dos tools: lo vemos en el paso 3.

IDEA CLAVE Un servidor MCP envuelve lo que tu agente necesita (tus datos, una API, un documento) para cualquier host.

thePower Tech School

Cómo se enchufa: un servidor, cuatro hosts

EL MISMO CONECTOR (MCP) · solo cambia el enchufe de cada host: un comando, un JSON, dos líneas de Python o una URL

Claude Code · un comando en la terminal

```
claude mcp add mi-servidor
  -- python server.py
```

STDIO · ✓ Connected

host_ollama.py (paso 5) · dos líneas de Python

```
StdioServerParameters(
  command=python, args=["server.py"])
```

STDIO · con llama3.1

mi-servidor

`server.py`

`consultar_pedido`
`convertir_precio`
`tienda://politica-devoluciones`

se escribe UNA vez

Tu equipo, en el repo · un JSON: .mcp.json

```
"mi-servidor": {"command": "uv",
  "args": ["run", …, "server.py"]}
```

STDIO · Claude Code pide aprobarlo

agente-tienda (S20) · una URL

```
Client("http://…:8000/mcp")
```

HTTP · el servidor en la nube

AL ENCHUFARLO, COMO UN USB:
el host pregunta qué es

`initialize` ¿quién eres? → `tools/list` ¿qué sabes hacer? → `tools/call` hazlo

IDEA CLAVE Escribes el servidor una vez; cada host solo necesita saber cómo enchufarlo.

thePower Tech School

Paso 1 · Objetivo

1

OBJETIVO DEL PASO

Crear el proyecto y un servidor MCP que arranca (aún sin tools)

DE LA TEORÍA

Slide 7: el SERVIDOR es el programa que construyes tú.

LO QUE VAMOS A VER

- uv init + uv add mcp[cli]
- MCPServer('mi-servidor')
- El handshake con 0 tools

ARCHIVO

- pasos/01_esqueleto.py

thePower Tech School

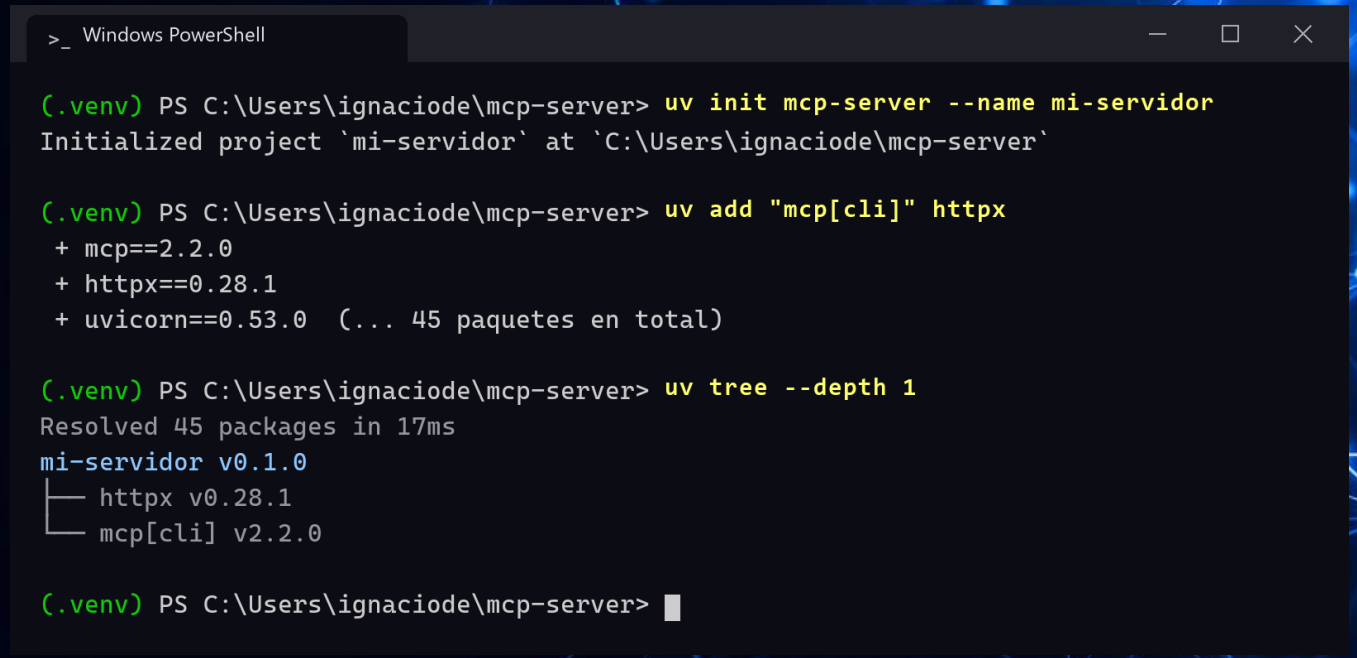
Paso 1 · Montar el proyecto y el esqueleto

QUÉ HACEMOS

- uv init: carpeta + pyproject.toml
- uv add "mcp[cli]": el SDK oficial
- server.py con la instancia del servidor
- Arranca por STDIO, todavía sin tools

CONCEPTO CLAVE

uv crea el entorno y fija las versiones. El SDK oficial de MCP para Python es un paquete más: mcp[cli].



FÍJATE EN

mcp 2.2.0: la versión importa. En la 2.x la clase ya no se llama FastMCP, se llama MCPServer.

thePower Tech School

Anatomía · el esqueleto en 4 líneas que importan

pasos/01_esqueleto.py

```python
import logging
import sys

# mcp 2.x: MCPServer (en la 1.x se llamaba FastMCP y se importaba de mcp.server.fastmcp)
from mcp.server.mcpserver import MCPServer

# Todo lo que no sea JSON-RPC va a stderr: stdout es el canal del protocolo
logging.basicConfig(stream=sys.stderr, level=logging.INFO, format="[mi-servidor] %(levelname)s %(message)s")

# La instancia del servidor. El nombre es como lo vera el host (Claude Code, VS Code...)
mcp = MCPServer("mi-servidor")

if __name__ == "__main__":
    logging.info("arrancando por STDIO (sin tools todavia)...")
    mcp.run()  # por defecto transport="stdio": lee de stdin, responde por stdout
```

1 (1)
2 (2)
3 (3)
4 (4)

1 Import del SDK 2.x
MCPServer; FastMCP era la 1.x

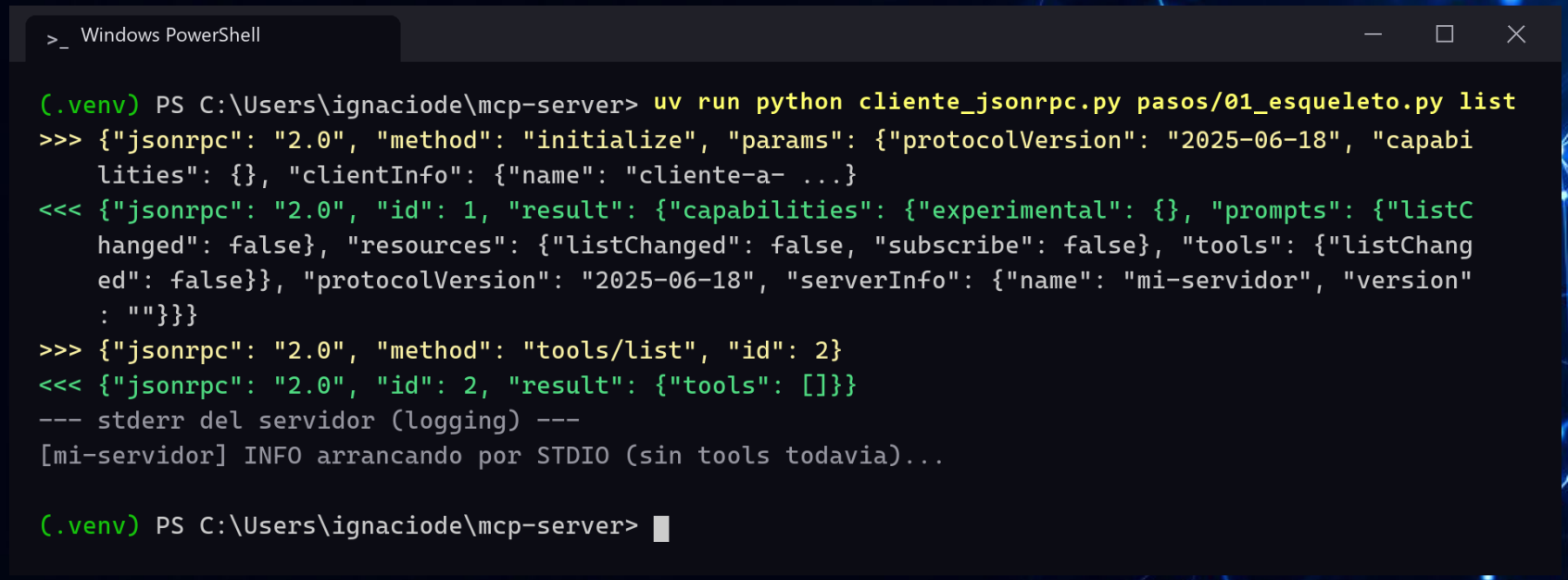
2 Logging a stderr
stdout es del protocolo

3 La instancia
el nombre que verá el host

4 mcp.run()
por defecto transporte STDIO

thePower Tech School

Zoom · el servidor responde... con 0 tools



SALIDA REAL **El handshake ya funciona**

- initialize: el cliente dice su versión y quién es
- El servidor contesta sus capacidades: tools, resources, prompts
- tools/list devuelve una lista vacía: aún no hay verbos
- El log sale por stderr, separado del JSON

thePower Tech School

Hallazgo real · el código de los tutoriales ya no funciona

mcp 1.x (tutoriales, muchos LLMs)

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("mi-servidor")
```

mcp 2.x (lo que instala uv hoy)

```
from mcp.server.mcpserver import MCPServer
mcp = MCPServer("mi-servidor")
```

```
ModuleNotFoundError: No module named 'mcp.server.fastmcp'. This is mcp 2.x, where FastMCP was renamed to MCPServer
```

Lección: comprueba la versión instalada antes de copiar código. Los decoradores @mcp.tool() y @mcp.resource() siguen igual; cambia el nombre de la clase.

thePower Tech School

Paso 1 · Pregunta a la clase

PREGUNTA A LA CLASE

Si el servidor no tiene ninguna tool, ¿para qué sirve arrancarlo ya?

Pensad en lo que acabamos de comprobar sin escribir ni una función.

thePower Tech School

Paso 1 · La respuesta

¿Para qué arrancarlo sin tools?

- Comprobamos el transporte: el JSON entra y sale por donde toca
- Comprobamos el SDK: versión, import y handshake
- Si luego falla una tool, ya sabemos que no es la conexión
- Es el 'hola mundo' de MCP: un paso pequeño y verificable

thePower
Tech School

Paso 1 · Lo que te llevas

- ✓ **uv + mcp[cli]** un proyecto reproducible en dos comandos
- ✓ **MCPServer** el nombre en la 2.x del SDK
- ✓ **mcp.run()** STDIO por defecto
- ✓ **Handshake** initialize y tools/list ya responden

SIGUIENTE Le damos al servidor su primer verbo: una tool que consulta una API real.

thePower Tech School

Paso 2 · Objetivo

2

OBJETIVO DEL PASO

Escribir la primera tool y ver la definición que genera el SDK

DE LA TEORÍA

Slide 9: escribir un buen docstring ES escribir tu API.

LO QUE VAMOS A VER

- @mcp.tool() sobre una función async
- Una API pública sin clave
- El inputSchema generado

ARCHIVO

- pasos/02_primera_tool.py

thePower Tech School

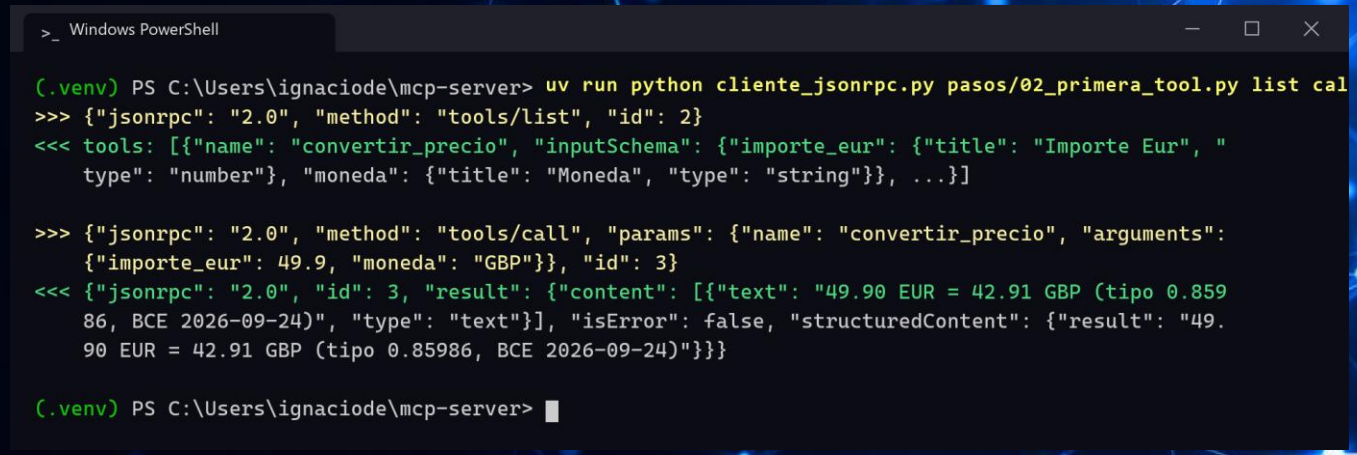
Paso 2 · Tu primera tool con @mcp.tool()

QUÉ HACEMOS

- Una función async con type hints
- Docstring: qué hace y cada parámetro
- Llama a Frankfurter (tipos del BCE)
- tools/list y tools/call desde el cliente

CONCEPTO CLAVE

El decorador convierte la función en tool: nombre, docstring y type hints pasan a ser la definición que lee el modelo.



FÍJATE EN

49,90 EUR = 42,91 GBP con el tipo real del BCE de ayer. La tool funciona antes de conectar ningún LLM.

thePower Tech School

Anatomía · de función Python a herramienta

pasos/02_primera_tool.py

```python
@mcp.tool()  # el decorador registra la funcion: nombre + docstring + type hints -> definicion de la tool
async def convertir_precio(importe_eur: float, moneda: str) -> str:
    """Convierte un importe en euros a otra moneda con el tipo de cambio oficial del BCE de hoy.

    Args:
        importe_eur: cantidad en euros, por ejemplo 49.90
        moneda: codigo ISO de la moneda destino, por ejemplo USD o GBP
    """
    url = f"https://api.frankfurter.dev/v1/latest?from=EUR&to={moneda.upper()}"
    async with httpx.AsyncClient(timeout=10) as cliente:   # async: no bloquea el servidor mientras espera la red
        datos = (await cliente.get(url)).json()
    tipo = datos["rates"][moneda.upper()]
    return f"{importe_eur:.2f} EUR = {importe_eur * tipo:.2f} {moneda.upper()} (tipo {tipo}, BCE {datos['date']})"
```

1 **@mcp.tool()**
registra la función en el servidor

2 **Firma con tipos**
float y str -> JSON Schema

3 **El docstring**
la descripción que lee el modelo

4 **async + httpx**
no bloquea mientras espera a la red

thePower Tech School

Zoom · lo que ve el modelo de tu tool

Windows PowerShell

```
(.venv) PS C:\Users\ignaciode\mcp-server> # tools/list -> la definicion que ve el modelo (generada por e
{
  "name": "convertir_precio",
  "description": "Convierte un importe en euros a otra moneda con el tipo de cambio oficial del BCE
    de hoy.\n\n    Args:\n        importe_eur: cantidad en euros, por ejemplo 49.90\n        moneda:
    codigo ISO de la moneda destino, por ejemplo USD o GBP\n    ",
  "inputSchema": {
    "properties": {
      "importe_eur": {
        "title": "Importe Eur",
        "type": "number"
      },
      "moneda": {
        "title": "Moneda",
        "type": "string"
      }
    },
    "required": [
      "importe_eur",
      "moneda"
    ],
    "type": "object",
    "title": "convertir_precioArguments"
  }
}

(.venv) PS C:\Users\ignaciode\mcp-server>
```

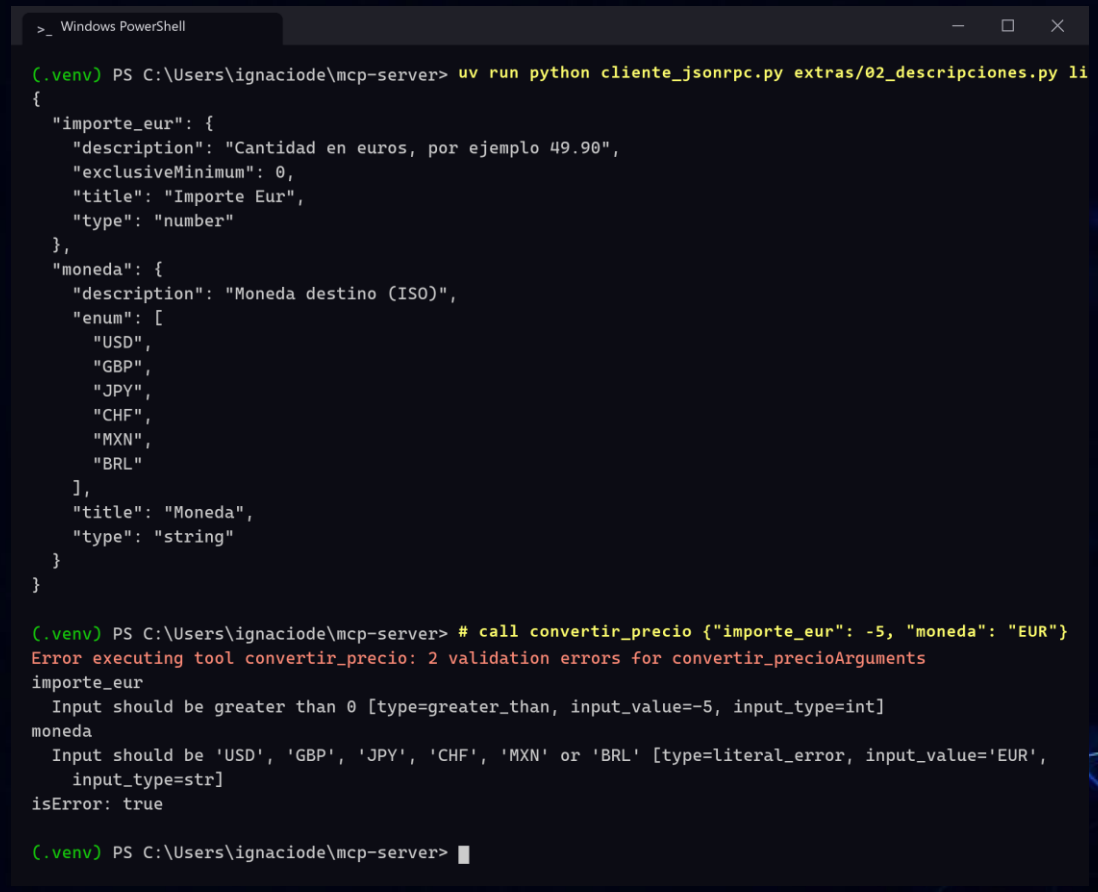
SALIDA REAL

Tus type hints, convertidos en JSON Schema

- name: el nombre de la función, tal cual
- description: el docstring ENTERO, Args incluidos
- inputSchema: tipos (number, string) y required
- Ojo: los parámetros no traen description propia

thePower Tech School

Variación · descripciones y límites dentro del esquema



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Annotated + Field(description=..., gt=0) y Literal con las monedas válidas.

QUÉ DEMUESTRA

Cada parámetro trae su descripción, un mínimo y un enum. Y la validación la hace el SDK: -5 y 'EUR' se rechazan con isError: true sin tocar tu código.

thePower Tech School

Paso 2 · Pregunta a la clase

PREGUNTA A LA CLASE

El modelo nunca ve tu código. Entonces, ¿cómo sabe cuándo llamar a convertir_precio?

Mirad otra vez qué viaja en tools/list.

thePower Tech School

Paso 2 · La respuesta

¿Cómo decide el modelo?

- Solo ve name, description e inputSchema
- Un nombre y un docstring vagos = una tool que no se usa
- Los tipos le dicen QUÉ pasar; el docstring, CUÁNDO
- Por eso escribir el docstring es escribir tu API

thePower
Tech School

Paso 2 · Lo que te llevas

- ✓ **@mcp.tool()** una función normal se vuelve herramienta
- ✓ **Type hints** se convierten en el inputSchema
- ✓ **Docstring** es la documentación que lee el modelo
- ✓ **Field y Literal** descripciones y validación gratis

SIGUIENTE Conectamos el servidor a un host de verdad: Claude Code.

thePower Tech School

Paso 3 · Objetivo

3

OBJETIVO DEL PASO

Registrar el servidor en Claude Code por STDIO y ver que conecta

DE LA TEORÍA

Slide 10: STDIO en local; nunca print() en stdout.

LO QUE VAMOS A VER

- claude mcp add + claude mcp list
- Scope local vs .mcp.json del proyecto
- Qué pasa con un print()

ARCHIVO

- claude mcp add

thePower Tech School

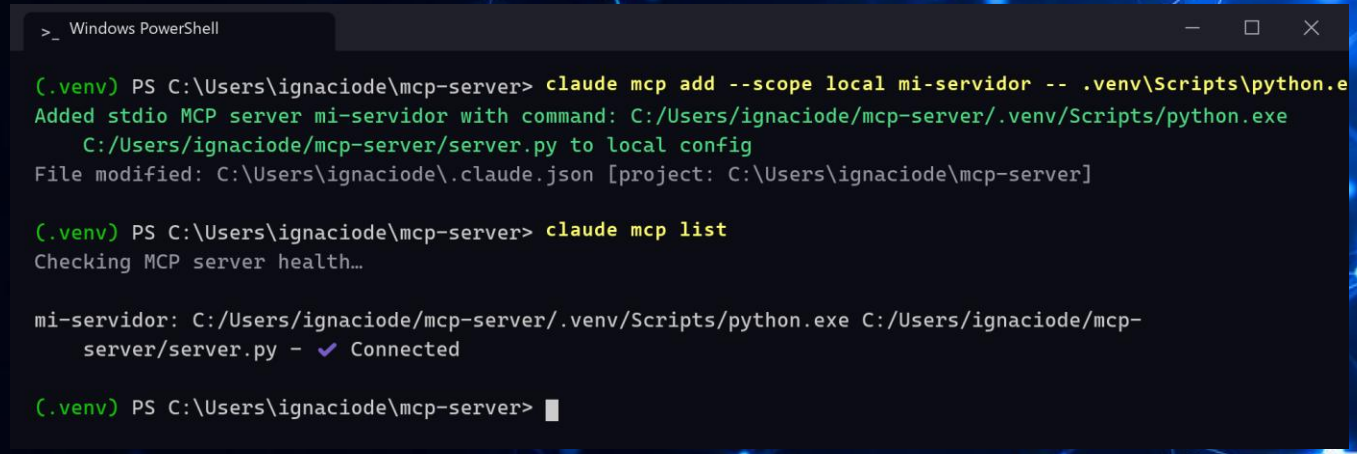
Paso 3 · Conectar el servidor a Claude Code (STDIO)

QUÉ HACEMOS

- claude mcp add: nombre + command + args
- El host arranca server.py como subproceso
- claude mcp list: comprueba que conecta
- Dentro de Claude Code: /mcp

CONCEPTO CLAVE

En STDIO el host lanza tu servidor y le habla por stdin/stdout. Sin red, sin puertos: solo un comando.



FÍJATE EN

✓*Connected: Claude Code arranca el servidor, hace el handshake y lo da por bueno.*

thePower Tech School

Zoom · .mcp.json: para el equipo, con permiso

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\mcp-server> claude mcp add --scope project mi-servidor -- .venv\Scripts\python
Added stdio MCP server mi-servidor with command: C:/Users/ignaciode/mcp-server/.venv/Scripts/python.exe
    C:/Users/ignaciode/mcp-server/server.py to project config
File modified: C:\Users\ignaciode\mcp-server\.mcp.json

(.venv) PS C:\Users\ignaciode\mcp-server> type .mcp.json
{
  "mcpServers": {
    "mi-servidor": {
      "type": "stdio",
      "command": "C:/Users/ignaciode/mcp-server/.venv/Scripts/python.exe",
      "args": [
        "C:/Users/ignaciode/mcp-server/server.py"
      ],
      "env": {}
    }
  }
}

(.venv) PS C:\Users\ignaciode\mcp-server> claude mcp list
Checking MCP server health…

mi-servidor: C:/Users/ignaciode/mcp-server/.venv/Scripts/python.exe C:/Users/ignaciode/mcp-
    server/server.py - □ Pending approval (run `claude` to approve)

(.venv) PS C:\Users\ignaciode\mcp-server>
```

SALIDA REAL

Pending approval

- --scope project escribe .mcp.json en el repo
- Cualquiera que clone el repo recibe la config
- Claude Code NO lo arranca sin tu aprobación
- Seguridad: un servidor es código que se ejecuta en tu PC

thePower Tech School

Zoom · lo que Claude Code hace por debajo: el handshake

Windows PowerShell

```
(.venv) PS C:\Users\ignaciode\mcp-server> # el handshake: initialize (cliente) -> capacidades (servidor)
>>> {"jsonrpc": "2.0", "method": "initialize", "params": {"protocolVersion": "2025-06-18",
    "capabilities": {}, "clientInfo": {"name": "cliente-a-pelo", "version": "1.0"}}, "id": 1}
<<< {
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "capabilities": {
      "experimental": {},
      "prompts": {
        "listChanged": false
      },
      "resources": {
        "listChanged": false,
        "subscribe": false
      },
      "tools": {
        "listChanged": false
      }
    },
    "protocolVersion": "2025-06-18",
    "serverInfo": {
      "name": "mi-servidor",
      "version": ""
    }
  }
}
>>> {"jsonrpc": "2.0", "method": "notifications/initialized"}

(.venv) PS C:\Users\ignaciode\mcp-server>
```

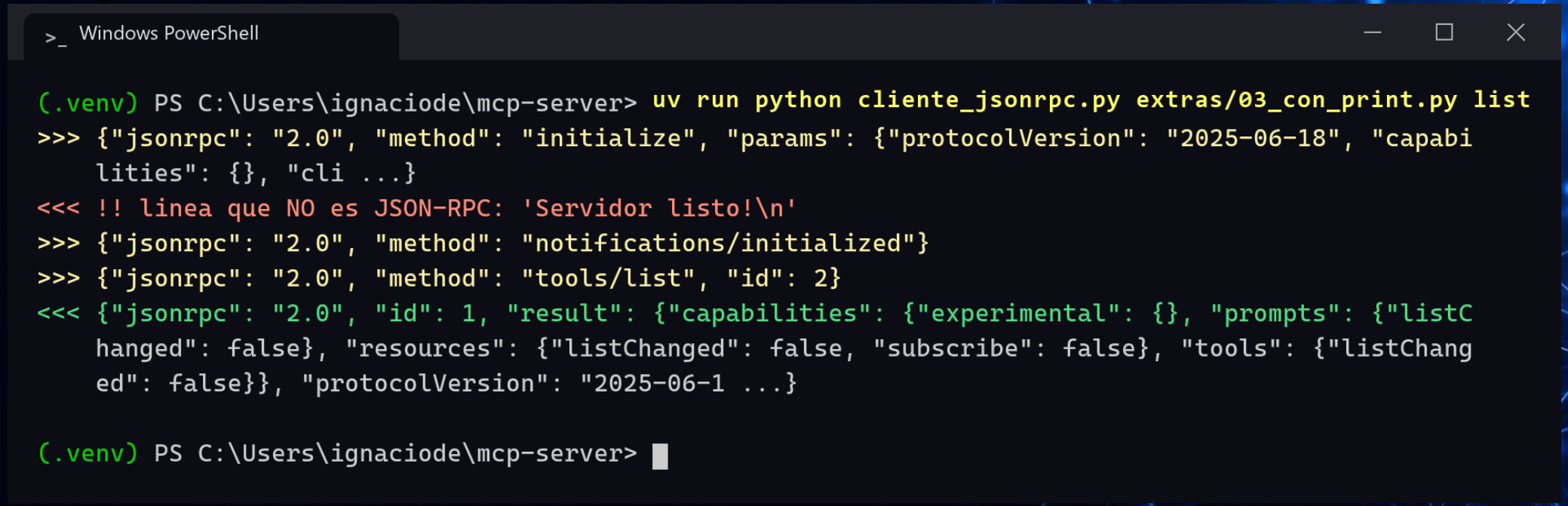
SALIDA REAL

initialize, en crudo

- El cliente propone versión de protocolo: 2025-06-18
- El servidor declara capacidades: tools, resources, prompts
- notifications/initialized: no lleva id, no espera respuesta
- Todo es JSON-RPC 2.0: una línea por mensaje

thePower Tech School

Variación · el error número uno: un print() en stdout



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Un print('¡Servidor listo!', flush=True) justo antes de mcp.run().

QUÉ DEMUESTRA

El cliente recibe '¡Servidor listo!' donde esperaba JSON. Y todo se desincroniza: a tools/list le llega la respuesta del id 1.

thePower Tech School

Paso 3 · Pregunta a la clase

PREGUNTA A LA CLASE

Si el SDK 2.x ya desvía los print() de dentro de las tools, ¿seguimos necesitando la regla?

Pensad en quién más puede escribir en stdout de ese proceso.

thePower Tech School

Paso 3 · La respuesta

¿Sigue valiendo la regla?

- Sí: el print antes de run() rompió el protocolo
- Librerías, otros SDK u otras versiones no te protegen
- logging a stderr además te da niveles y formato
- Y el host (Claude Code) guarda ese stderr como log

thePower
Tech School

Paso 3 · Lo que te llevas

- ✓ **claude mcp add** nombre + command + args, por STDIO
- ✓ **Scopes** local para ti, project (.mcp.json) con aprobación
- ✓ **claude mcp list** health check: ✓ Connected
- ✓ **stdout es sagrado** el log, siempre a stderr

SIGUIENTE Ampliamos el servidor: un resource para leer y una segunda tool que valida lo que entra.

thePower Tech School

Paso 4 · Objetivo

4

OBJETIVO DEL PASO

Añadir un resource de solo lectura y una segunda tool que valida

DE LA TEORÍA

Slide 8: tools = verbos (decide el modelo), resources = datos (decide la app).

LO QUE VAMOS A VER

- @mcp.resource('tienda://...')
- consultar_pedido de la Sesión 5
- Errores claros, no excepciones

ARCHIVO

- server.py

thePower Tech School

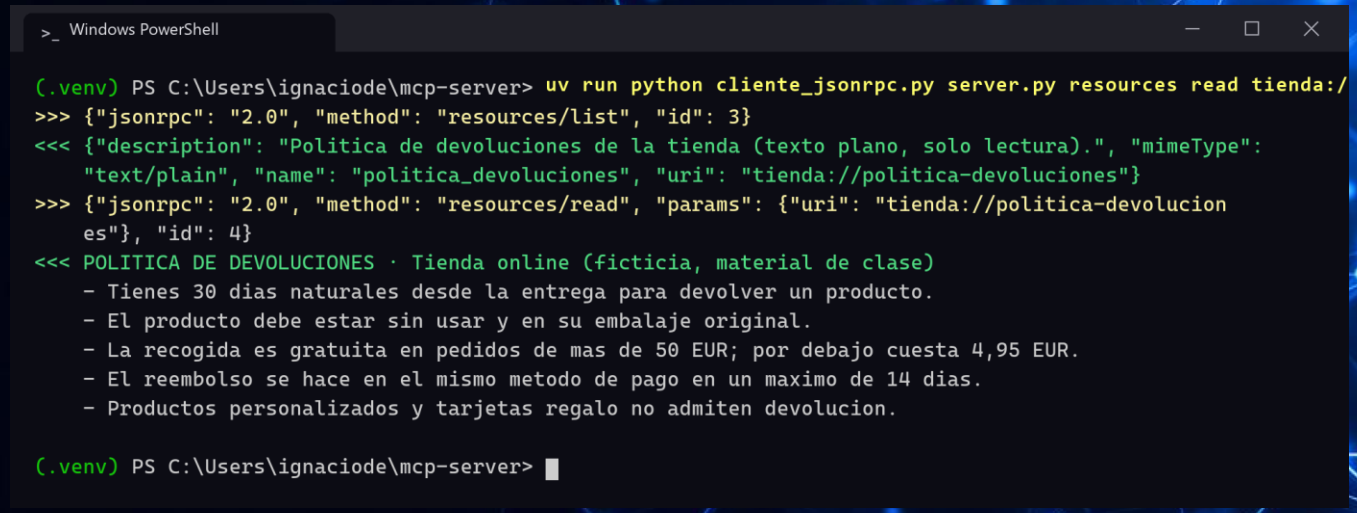
Paso 4 · Añadir un resource y una segunda tool

QUÉ HACEMOS

- Resource: política de devoluciones por URI
- Tool: consultar_pedido (la de la Sesión 5)
- Validar entradas: 4 cifras, moneda soportada
- Errores con mensaje útil para el modelo

CONCEPTO CLAVE

Un resource se LEE (como un GET, por URI). Una tool se EJECUTA (puede tener efectos). Mismo servidor, dos primitivas.



FÍJATE EN

resources/list trae uri, nombre y mimeType; resources/read devuelve el texto tal cual.

thePower Tech School

Anatomía · la segunda tool: validar antes de responder

server.py

```python
def consultar_pedido(numero: str) -> dict:
    """Devuelve el estado, el transportista, el dia de llegada y el importe en euros de un pedido.

    Args:
        numero: numero de pedido de 4 cifras, por ejemplo "1234"
    """
    numero = numero.strip().lstrip("#")
    if not (numero.isdigit() and len(numero) == 4):
        log.warning("numero de pedido invalido: %r", numero)
        return {"error": "El numero de pedido debe tener 4 cifras, por ejemplo 1234"}
    log.info("consultar_pedido(%s)", numero)
    return PEDIDOS.get(numero, {"error": f"No existe el pedido {numero}"})
```

1

Firma y docstring

dice el formato esperado: 4 cifras

2

Limpiar y validar

quita '#', rechaza '12a'

3

Respuesta

dict: el SDK lo pasa a JSON

thePower Tech School

Anatomía · un resource es una función con URI

server.py

```python
@mcp.resource("tienda://politica-devoluciones")
def politica_devoluciones() -> str:
    """Politica de devoluciones de la tienda (texto plano, solo lectura)."""
    log.info("leyendo resource politica-devoluciones")
    with open(__file__.replace("server.py", "datos/politica_devoluciones.txt"), encoding="utf-8") as f:
        return f.read()
```

1. **@mcp.resource(uri)** — la dirección del dato
2. **Docstring** — la descripción en resources/list
3. **Solo lectura** — devuelve texto, no cambia nada

thePower Tech School

Zoom · cinco llamadas, cinco respuestas útiles

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\mcp-server> uv run python cliente_jsonrpc.py server.py call ... (5 llamadas)
>>> {"jsonrpc": "2.0", "method": "tools/call", "params": {"name": "convertir_precio", "arguments":
     {"importe_eur": 120, "moneda": "MXN"}}, "id": 5}
<<< 120.00 EUR = 2398.54 MXN (tipo 19.9878, BCE 2026-09-24)
>>> {"jsonrpc": "2.0", "method": "tools/call", "params": {"name": "consultar_pedido", "arguments":
     {"numero": "#5678"}}, "id": 6}
<<< {  "estado": "en preparacion",  "transportista": "-",  "llegada": "la semana que viene",
    "importe_eur": 120.0 }
>>> {"jsonrpc": "2.0", "method": "tools/call", "params": {"name": "consultar_pedido", "arguments":
     {"numero": "12a"}}, "id": 7}
<<< {  "error": "El numero de pedido debe tener 4 cifras, por ejemplo 1234" }
>>> {"jsonrpc": "2.0", "method": "tools/call", "params": {"name": "consultar_pedido", "arguments":
     {"numero": "9999"}}, "id": 8}
<<< {  "error": "No existe el pedido 9999" }
>>> {"jsonrpc": "2.0", "method": "tools/call", "params": {"name": "convertir_precio", "arguments":
     {"importe_eur": 10, "moneda": "bitcoin"}}, "id": 9}
<<< Error: moneda 'BITCOIN' no soportada. Usa una de: BRL, CHF, GBP, JPY, MXN, USD
--- stderr del servidor (logging) ---
[mi-servidor] INFO arrancando por STDIO...
[mi-servidor] INFO leyendo resource politica-devoluciones
[mi-servidor] INFO convertir_precio(120.0, MXN)
[mi-servidor] INFO HTTP Request: GET https://api.frankfurter.dev/v1/latest?from=EUR&to=MXN "HTTP/1.1 200
                                 OK"
[mi-servidor] INFO consultar_pedido(5678)
[mi-servidor] WARNING numero de pedido invalido: '12a'
[mi-servidor] INFO consultar_pedido(9999)

(.venv) PS C:\Users\ignaciode\mcp-server>
```

SALIDA REAL

La validación en acción

- 120 EUR = 2398,54 MXN con el tipo real
- '#5678' se limpia y funciona
- '12a' -> mensaje que dice CÓMO arreglarlo
- 'bitcoin' -> la lista de monedas válidas
- El log a stderr cuenta todo: INFO y WARNING

thePower Tech School

Paso 4 · Pregunta a la clase

PREGUNTA A LA CLASE

La política de devoluciones, ¿debería ser un resource o una tool?

Pensad en quién decide leerla y si cambia algo al leerla.

thePower Tech School

Paso 4 · La respuesta

¿Resource o tool?

- Resource: son datos fijos, solo lectura, con una URI
- La app (o tú con @) decide cuándo meterlos en contexto
- Si hubiera que BUSCAR dentro, sería una tool: buscar_politica(tema)
- Regla: si tiene efectos o parámetros, tool

thePower
Tech School

Paso 4 · Lo que te llevas

- ✓ **@mcp.resource** datos por URI, solo lectura
- ✓ **Segunda tool** reutilizamos consultar_pedido de la S5
- ✓ **Validar** limpiar, comprobar y explicar el error
- ✓ **Errores útiles** el modelo puede corregirse con ellos

SIGUIENTE Lo probamos de punta a punta con un modelo de verdad decidiendo qué tool usar.

thePower Tech School

Paso 5 · Objetivo

5

OBJETIVO DEL PASO

Probar de punta a punta con un LLM y empaquetar

DE LA TEORÍA

Slide 14: initialize -> tools/list -> tools/call.

LO QUE VAMOS A VER

- Un host casero con llama3.1
- Qué pasa cuando hay que encadenar 2 tools
- README listo para compartir

ARCHIVO

- host_ollama.py

thePower Tech School

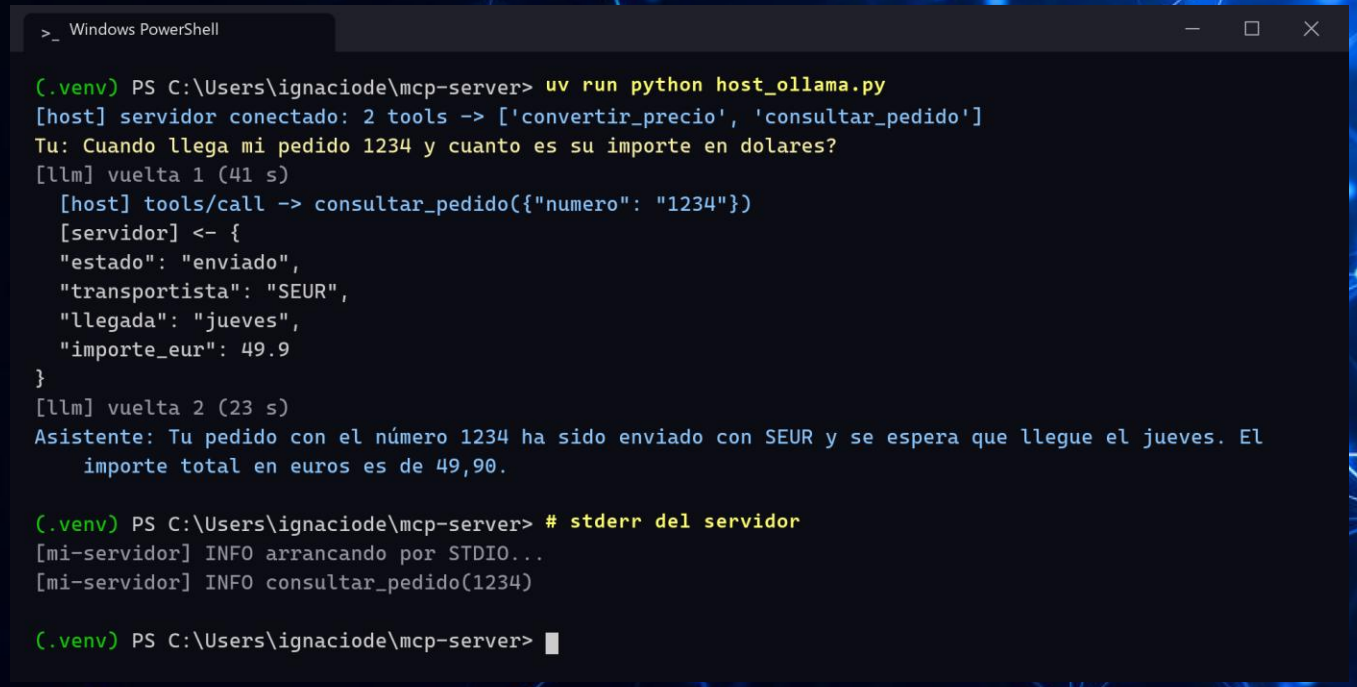
Paso 5 · Probar de punta a punta y empaquetar

QUÉ HACEMOS

- host_ollama.py: un host MCP en ~50 líneas
- tools/list -> se las damos a llama3.1
- Si pide una tool -> tools/call al servidor
- README: instalar, conectar, extender

CONCEPTO CLAVE

Un host es esto: pasa las definiciones al LLM y ejecuta las tools que pide. Lo mismo que Claude Code, en pequeño.



FÍJATE EN

Acertó con consultar_pedido... pero se olvidó de los dólares. Contestó solo en euros.

thePower Tech School

Anatomía · un host MCP en pequeño

host_ollama.py

```python
    async with Client(servidor) as mcp:
        tools = (await mcp.list_tools()).tools                                          # 1
        print(f"[host] servidor conectado: {len(tools)} tools -> {[t.name for t in tools]}")
        # El inputSchema que genero el SDK ES el formato de function calling del LLM
        herramientas = [{"type": "function", "function": {"name": t.name, "description": t.description,   # 2
                                                          "parameters": t.input_schema}} for t in tools]
        mensajes = [{"role": "system", "content": "Eres el asistente de una tienda online. Responde en espanol, breve. "
                                                  "Usa las herramientas cuando haga falta. "
                                                  "Si la pregunta tiene varias partes, usa una herramienta para cada parte "
                                                  "antes de responder (p. ej. primero el pedido y luego convertir su importe)."},
                    {"role": "user", "content": pregunta}]
        print(f"Tu: {pregunta}")
        async with httpx.AsyncClient(timeout=300) as http:
            for vuelta in range(4):                                                     # 3 tope de vueltas: nunca un bucle infinito
                t0 = time.time()
                r = await http.post(OLLAMA, json={"model": MODELO, "messages": mensajes, "tools": herramientas,
                                                  "stream": False, "keep_alive": 0,
                                                  "options": {"temperature": 0, "num_predict": 200}})
                msg = r.json()["message"]
                print(f"[llm] vuelta {vuelta + 1} ({time.time() - t0:.0f} s)")
                mensajes.append(msg)
                if not msg.get("tool_calls"):
                    print(f"Asistente: {msg['content']}")
                    return
                for llamada in msg["tool_calls"]:
                    nombre, args = llamada["function"]["name"], llamada["function"]["arguments"]
                    print(f"  [host] tools/call -> {nombre}({json.dumps(args, ensure_ascii=False)})")
                    res = await mcp.call_tool(nombre, args)                             # 4
                    texto = "\n".join(c.text for c in res.content if getattr(c, "text", None))
                    print(f"  [servidor] <- {texto}")
                    mensajes.append({"role": "tool", "content": texto, "tool_name": nombre})
```

1. **tools/list**
   el host descubre las tools

2. **inputSchema = parameters**
   el formato de function calling

3. **Tope de vueltas**
   nunca un bucle infinito

4. **tools/call**
   el host ejecuta, el LLM solo pide

thePower Tech School

Zoom · segundo intento: dos tools a la vez… y a ciegas

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\mcp-server> uv run python host_ollama.py   # con system prompt 'una tool por cada part
[host] servidor conectado: 2 tools -> ['convertir_precio', 'consultar_pedido']
Tu: Cuando llega mi pedido 1234 y cuanto es su importe en dolares?
[llm] vuelta 1 (38 s)
  [host] tools/call -> consultar_pedido({"numero": "1234"})
  [servidor] <- {
  "estado": "enviado",
  "transportista": "SEUR",
  "llegada": "jueves",
  "importe_eur": 49.9
}
  [host] tools/call -> convertir_precio({"importe_eur": "importe en euros", "moneda": "USD"})
  [servidor] <- Error executing tool convertir_precio: 1 validation error for convertir_precioArguments
importe_eur
  Input should be a valid number, unable to parse string as a number [type=float_parsing, input_value='importe
    en euros', input_type=str]
[llm] vuelta 2 (35 s)
Asistente: Lo siento, parece que hubo un error en la conversión del precio. El importe de tu pedido en euros es
    49,9. Para obtener el importe en dólares, necesitaría saber el tipo de cambio actual. ¿Quieres que lo
    averigüe?

(.venv) PS C:\Users\ignaciode\mcp-server>
```

SALIDA REAL

El esquema paró el golpe

- Pidió consultar_pedido y convertir_precio EN PARALELO
- No sabía el importe: mandó el texto 'importe en euros'
- El SDK validó el tipo (number) y devolvió el error
- Sin validación, eso llega a tu API o a tu base de datos

thePower Tech School

Variación · una tool pensada para la tarea

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\mcp-server> uv run python host_ollama.py --servidor extras/server_tarea.py
[host] servidor conectado: 3 tools -> ['convertir_precio', 'consultar_pedido', 'importe_pedido_en']
Tu: Cuando llega mi pedido 1234 y cuanto es su importe en dolares?
[llm] vuelta 1 (44 s)
  [host] tools/call -> consultar_pedido({"numero": "1234"})
  [servidor] <- {
  "estado": "enviado",
  "transportista": "SEUR",
  "llegada": "jueves",
  "importe_eur": 49.9
}
  [host] tools/call -> convertir_precio({"importe_eur": "importe_pedido_en", "moneda": "USD"})
  [servidor] <- Error executing tool convertir_precio: 1 validation error for convertir_precioArguments
importe_eur
  Input should be a valid number, unable to parse string as a number [type=float_parsing,
    input_value='importe_pedido_en', input_type=str]
[llm] vuelta 2 (46 s)
Asistente: Lo siento, pero parece que hubo un error en la conversión del precio. El importe del pedido 1234 en
    euros es 49,9. Para obtener el importe en dólares, necesitaría el tipo de cambio actual, que no tengo acceso
    a la fecha de hoy.

(.venv) PS C:\Users\ignaciode\mcp-server>
```

VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

extras/server_tarea.py añade importe_pedido_en(numero, moneda): resuelve la pregunta en una sola llamada.

QUÉ DEMUESTRA

llama3.1 la ignoró y repitió el error. Con un modelo de 8B, más tools = más confusión. Diseña pocas tools, claras, y valida siempre.

thePower Tech School

Recap: de la teoría al código

Teoría	En el código	Lo que vimos de verdad
Slide 6 · Servidor	MCPServer('mi-servidor')	en mcp 2.x ya no es FastMCP
Slide 7 · Tools y resources	@mcp.tool() / @mcp.resource()	2 tools + política por URI
Slide 8 · Docstring = API	description + inputSchema	los Args no van por parámetro
Slide 9 · STDIO y print()	logging a stderr	un print desincroniza el JSON-RPC
Slide 10 · El ciclo	cliente_jsonrpc.py	initialize, list y call en crudo
Slide 11 · Validar	Field, Literal y checks	el esquema frenó a llama3.1

thePower
Tech School

Paso 5 · Lo que te llevas

- ✓ **Un servidor, muchos hosts** Claude Code lo conecta, llama3.1 lo usa
- ✓ **Host = descubrir + ejecutar** el LLM solo pide, el host llama
- ✓ **Modelos pequeños** encadenan mal: pocas tools y claras
- ✓ **Validación** tu red de seguridad ante argumentos inventados

SIGUIENTE Deberes: tu primer servidor con una tool y, el reto, 2 tools + 1 resource conectados a Claude Code.

thePower Tech School

Ejercicio 1 · Construye tu propio servidor MCP

QUÉ HACER

MEDIO · 2 h

1. **Proyecto con uv + mcp[cli]**
   comprueba la versión: MCPServer en la 2.x

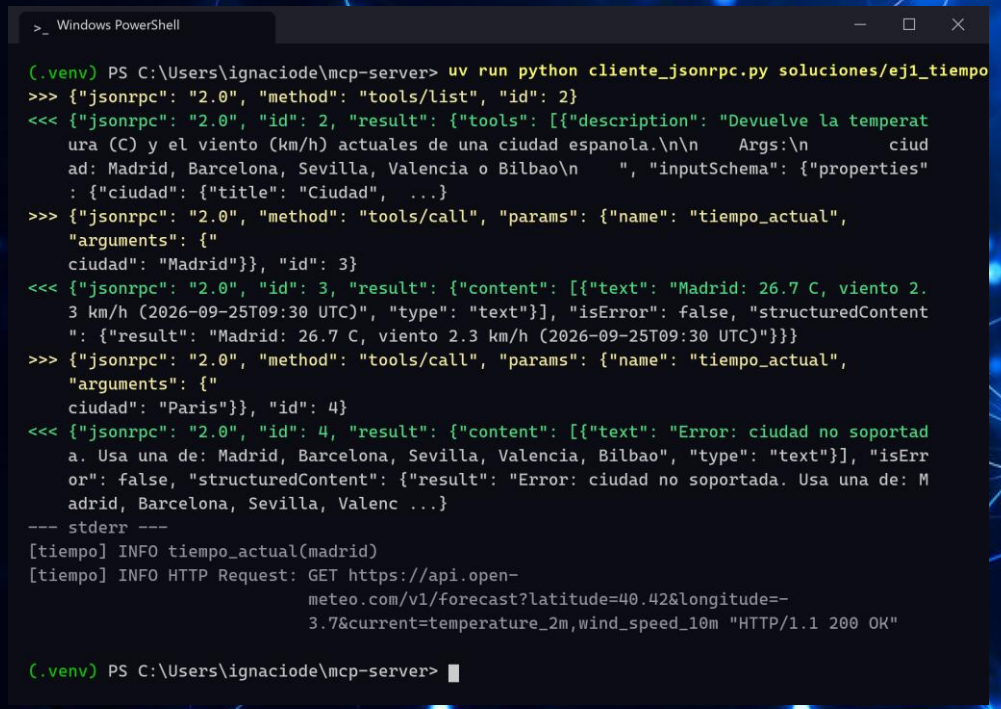
2. **UNA tool útil con @mcp.tool()**
   la solución: el tiempo con Open-Meteo, sin clave

3. **Conéctala por STDIO**
   claude mcp add + claude mcp list

4. **Úsala**
   tools/list y una llamada real; logging a stderr

ENTREGABLE

Repositorio (server.py + config) + captura del agente usando la tool.



ASÍ DEBE VERSE (SOLUCIÓN REAL)

Madrid: 26,7 C y viento 2,3 km/h en directo; 'París' devuelve la lista de ciudades válidas.

thePower Tech School

Ejercicio 2 · Reto: 2 tools + 1 resource en Claude Code

RETO

QUÉ CONSTRUIR

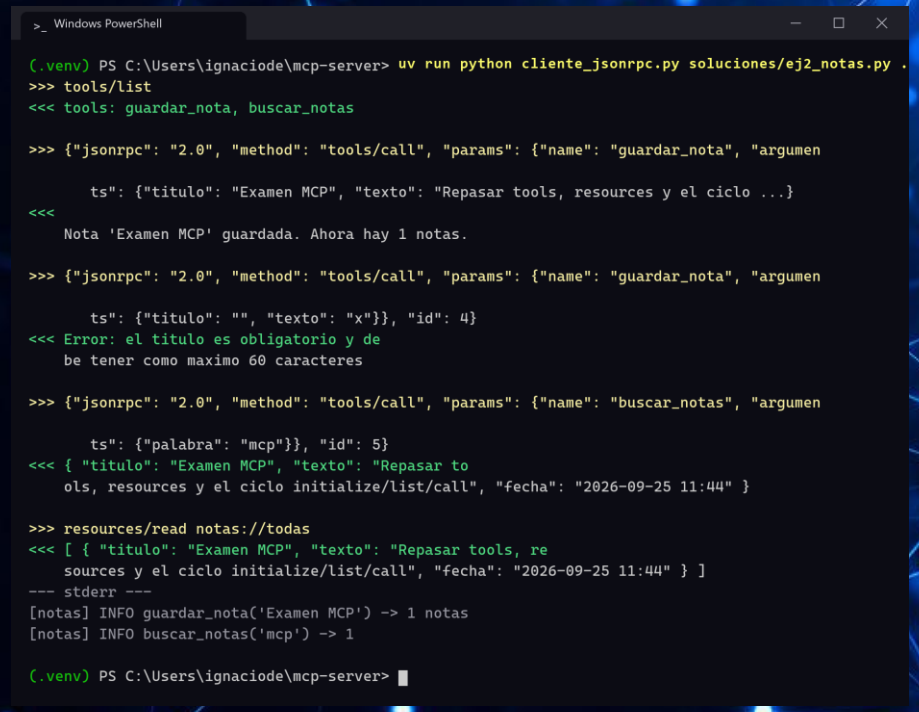
1. **Un dominio tuyo**
   la solución: un cuaderno de notas en JSON

2. **2 tools, una con efectos**
   guardar_nota y buscar_notas, con validación

3. **1 resource por URI**
   notas://todas, solo lectura

4. **Conectado + README**
   logging a stderr y cómo extenderlo

ENTREGABLE
Servidor funcionando en Claude Code + README. Solución en soluciones/ej2_notas.py.



LA SOLUCIÓN, EJECUTADA
Guarda la nota, rechaza un título vacío, la encuentra buscando 'mcp' y el resource la lista.

thePower Tech School

Test de repaso · 1/10

TEST · ELIGE UNA RESPUESTA

Con 3 clientes de IA y 10 herramientas, ¿cuántas piezas hacen falta con MCP en lugar de integraciones a medida?

- **A** 30 conectores, uno por cada pareja de cliente y herramienta
- **B** 13 piezas: cada cliente y cada herramienta se implementan una vez
- **C** 10 servidores, pero hay que reescribirlos para cada cliente
- **D** 3 conectores, uno por cliente, que ya incluyen todas las herramientas

thePower Tech School

Test de repaso · 2/10

TEST · ELIGE UNA RESPUESTA

En la versión 2.x del SDK oficial de MCP para Python, ¿cómo se llama la clase con la que se crea el servidor?

A FastMCP, que se importa del módulo fastmcp del SDK

B MCPClient, la misma clase que usa el host para conectarse

C Server, que obliga a escribir el JSON Schema de cada tool a mano

D MCPServer; FastMCP era el nombre que tenía en la versión 1.x

thePower Tech School

Test de repaso · 3/10

TEST · ELIGE UNA RESPUESTA

¿Por qué no se debe usar print() en un servidor MCP que funciona con el transporte STDIO?

A. Porque stdout es el canal de los mensajes JSON-RPC y se corrompe

B. Porque print es más lento que logging y el host corta por tiempo

C. Porque el SDK prohíbe print y lanza una excepción al arrancar

D. Porque lo impreso se envía al modelo como si fuera una tool nueva

thePower Tech School

Test de repaso · 4/10

TEST · ELIGE UNA RESPUESTA

En la prueba real, ¿qué pasó al poner un print() justo antes de arrancar el servidor con mcp.run()?

A. Nada: el SDK 2.x desvía a stderr cualquier print del proceso

B. El servidor se negó a arrancar y mostró un error de sintaxis

C. El cliente recibió texto no JSON y las respuestas llegaron un turno tarde

D. El host aprobó el servidor, pero ocultó todas sus tools

thePower Tech School

Test de repaso · 5/10

TEST · ELIGE UNA RESPUESTA

¿Qué información de una tool MCP ve el modelo para decidir cuándo llamarla y con qué argumentos?

A. Solo el nombre, la descripción y el esquema de entrada (inputSchema)

B. El código fuente de la función y las APIs a las que llama por dentro

C. Únicamente el nombre de la función y el número de parámetros

D. El historial de llamadas anteriores que guarda el propio servidor

thePower Tech School

Test de repaso · 6/10

TEST · ELIGE UNA RESPUESTA

Con el SDK 2.x, ¿qué ocurre con la sección Args del docstring de una tool al generar su definición?

A. Se reparte como description de cada parámetro en el inputSchema

B. Se descarta y el modelo no la ve en ningún sitio

C. Se convierte en un enum con los valores permitidos de cada parámetro

D. Va entera en la description general; Field la pone por parámetro

thePower Tech School

Test de repaso · 7/10

TEST · ELIGE UNA RESPUESTA

¿Qué estado mostró el chequeo de salud al registrar el servidor con el ámbito de proyecto (.mcp.json)?

A Connected, porque el fichero viene en el repositorio y es de confianza

B Pending approval: el host no lo arranca hasta que lo apruebas

C Failed to connect, porque el ámbito de proyecto exige usar HTTP

D Disabled, porque solo se permite un servidor MCP por carpeta

thePower Tech School

Test de repaso · 8/10

TEST · ELIGE UNA RESPUESTA

¿Cuándo conviene exponer algo como resource en un servidor MCP y no como tool?

A Cuando el modelo debe decidir por sí solo si lo usa y con qué parámetros

B Cuando la operación escribe en disco o envía un correo electrónico

C Cuando es un dato de solo lectura, sin efectos, identificado por una URI

D Cuando hay que buscar dentro de un documento largo por un tema concreto

Test de repaso · 9/10

TEST · ELIGE UNA RESPUESTA

En el segundo intento con llama3.1, ¿qué frenó el argumento importe_eur con el texto «importe en euros»?

A. El propio modelo, que corrigió el argumento antes de enviarlo

B. El tope de cuatro vueltas que tenía el host casero

C. La API de Frankfurter, que devolvió un error al recibir el texto

D. La validación del esquema: el SDK exigía un number y lo rechazó

thePower Tech School

Test de repaso · 10/10

TEST · ELIGE UNA RESPUESTA

En el host casero con llama3.1, ¿quién ejecuta realmente la tool cuando el modelo decide usarla?

A El host, que hace tools/call al servidor; el modelo solo la pide

B El modelo, que se conecta directamente al servidor por STDIO

C Ollama, que ejecuta el código Python de la tool en su propio proceso

D El cliente a pelo, que traduce la petición al formato de function calling

thePower Tech School